Data Science Capstone Report

SpaceX Falcon 9 First-Stage Landing Success Analysis & Prediction

SpaceX REST API | Web Scraping | EDA | SQL | Folium | Plotly Dash | ML Classification

GitHub repository: https://github.com/cho1eeee/spacex-capstone

Final deliverable: this report PDF (slide format) · October 2026

Executive Summary: Methodology and Key Results

90	66.7%	88.9%	4+1
Falcon 9 launch records	Overall landing success rate	Best-model test accuracy	Classification models compared

Method pipeline: SpaceX API data collection → web-scraping cross-validation → data wrangling (label engineering + one-hot encoding) → EDA visualization → SQL analysis → Folium maps / Plotly Dash interactive dashboard → multi-model classification.

Core question: Can first-stage booster recovery success be predicted from quantifiable features? Successful recovery cuts the cost per launch from ~\$283M (new booster) to ~\$62M.

Key results:

- 60 of 69 landing attempts succeeded (87.0%)
- KSC LC-39A has the highest success rate (77.3%)
- VLEO orbits carry the heaviest average payload (~15.3 t)
- Decision Tree is the best model: 88.9% test accuracy

Introduction: Background and Problem Statement

Background. SpaceX has disrupted the economics of space launch through booster reusability. The Falcon 9 first stage can return to land or to a droneship and fly again — the fundamental reason its pricing is far below traditional expendable rockets.

Problem statement. If first-stage recovery success could be predicted from launch parameters (payload mass, orbit type, launch site, booster state, etc.), it would:

- Inform booster-reuse scheduling and insurance pricing;
- Identify the key factors driving successful recovery.

Data science framing. Model it as a binary classification problem:

Class 1 = successful landing | Class 0 = failed landing

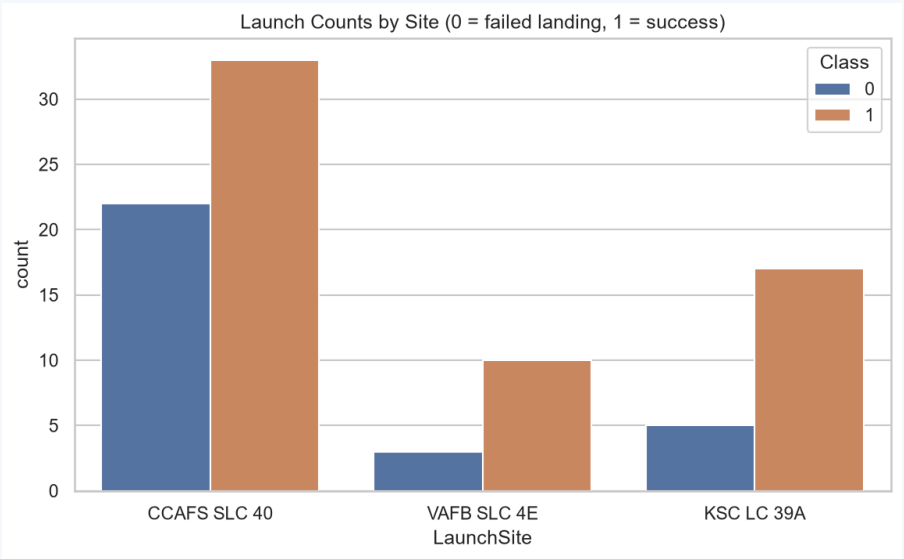
Data sources: SpaceX REST API v4 (launches, rockets, launchpads, payloads endpoints), supplemented by web scraping of public launch lists for cross-validation.

Time span: all 90 Falcon 9 launches from 2010–2020 (covering the full landing-attempt era).

Data Collection ①: SpaceX REST API Calls

POST /v4/launches/query (paginated query) → GET /v4/rockets (booster models) → GET /v4/launchpads (site coordinates) → GET /v4/payloads (payload mass/orbit) → pandas merge spacex_launch_data.csv

- **Core call:** send a JSON query body to `/launches/query`, page through all launches, and populate the related rocket / launchpad / payloads / cores.landpad records.
- Landing outcomes are derived from each core's `landing_attempt` + `landing_success`.
- The official API origin was down at collection time (522), so the pipeline **automatically fell back** to a hosted mirror snapshot of the same API data, cross-validated against the **live Launch Library 2 API** (821 cumulative Falcon 9 launches).



Collection results: launch/landing counts across the three launch sites

🔗 GitHub: https://github.com/cho1eeee/spacex-capstone — scripts/01_collect_data.py

Data Collection ②: Web Scraping (requests + BeautifulSoup)

requests.get(url) → Raw HTML page (169,000 characters) → BeautifulSoup find_all card parsing → Structured DataFrame → spacex_web_scraped.csv

- Scraped launch cards from a public spaceflight launch-list site (NextSpaceFlight.com): mission name, rocket model, launch provider, and launch location.
- Parsed **18** launch records, 5 of them SpaceX, covering Vandenberg SFB and Cape Canaveral SFS — **corroborating** the launch sites in the API dataset: **VAFB SLC-4E / CCAFS SLC-40 / KSC LC-39A**.
- Scraping is used for **cross-validation only**, not as a replacement for the primary dataset, keeping modeling features consistent.

```
Mission · Rocket · Provider · Location
Starlink Group 15-16 · Falcon 9 Block 5 · SpaceX · Vandenberg SFB, USA
Starlink Group 15-21 · Falcon 9 Block 5 · SpaceX · Vandenberg SFB, USA
NROL-195 · Falcon 9 Block 5 · SpaceX · Cape Canaveral SFS, USA
… 18 records in total (100% parse success, fields correctly paired)
```

Preview of scraping results (spacex_web_scraped.csv)

🔗 GitHub: https://github.com/cho1eeee/spacex-capstone — scripts/02_scrape_wikipedia.py

Data Wrangling Methods

1. Label engineering

- `Outcome` (e.g. True ASDS / False Ocean / None None) → binary label **Class** (1 success / 0 failure), plus a **LandingAttempt** flag.

2. Missing-value handling

- Missing payload mass → imputed with the mean of the same booster model; missing LandingPad → "Unknown".

3. Types and dates

- date_utc → datetime, deriving a **Year** feature for trend analysis.

4. Feature encoding

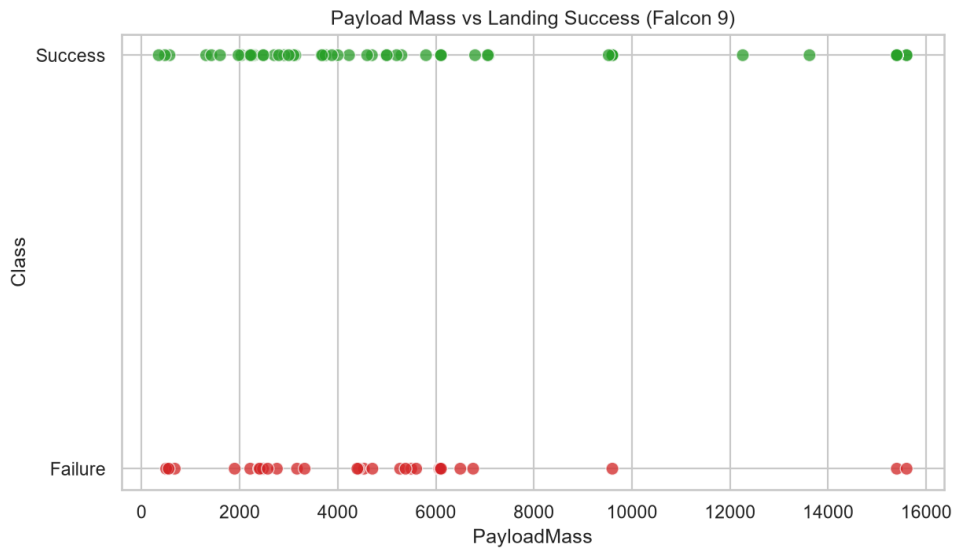
- **One-hot encoding** of Orbit / LaunchSite / LandingPad / Serial, producing an 82-dimensional feature matrix.
- Physical features retained: PayloadMass, Flights (reuse count), GridFins, Reused, Legs, Block, ReusedCount.

Outputs:

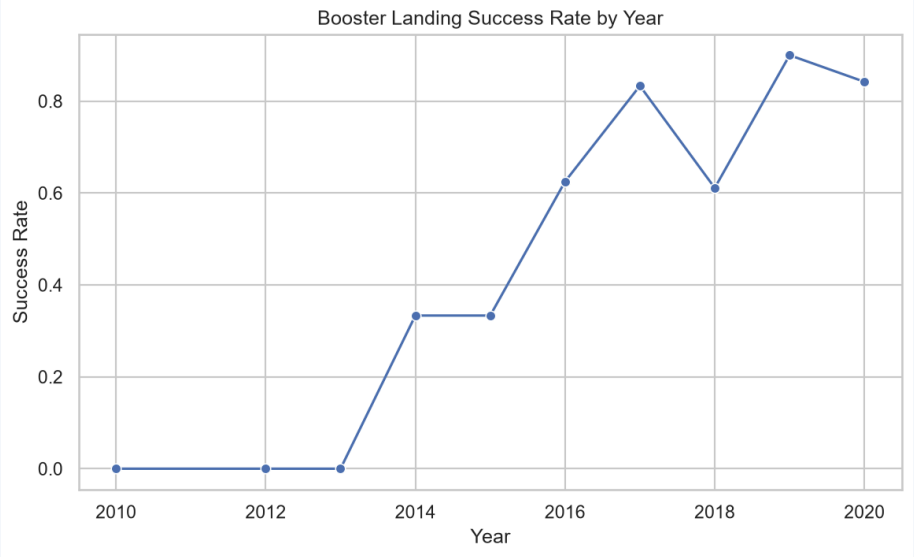
- `dataset_part_2.csv`: labeled cleaned data (90×20)
- `dataset_part_3.csv`: ML feature matrix (90×82)

🔗 GitHub: https://github.com/cho1eeee/spacex-capstone — scripts/03_clean_data.py · data/dataset_part_2.csv, dataset_part_3.csv

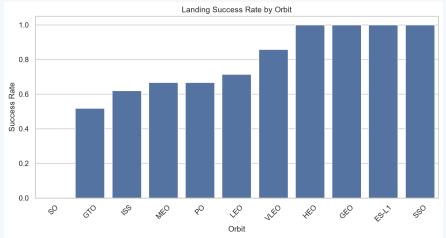
Exploratory Data Analysis (EDA): Plots and Purpose



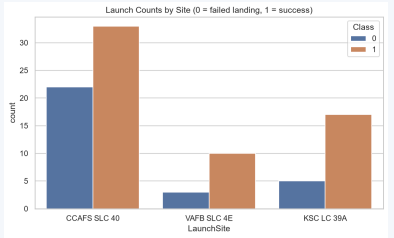
① Payload mass vs landing outcome (scatter)



② Success rate trend by year (line)



③ Success rate by orbit (bar)



④ Launch/landing counts by site (bar)

Findings: the success rate rose from 0% in 2013 to 84–90% in 2019–2020; heavy payloads (>10 t, VLEO/Starlink) are recovered with high probability too; increased reuse has not significantly reduced the success rate.

Exploratory Analysis with SQL (SQLite)

Cleaned data loaded into SQLite (SPACEXTBL); 8 query groups executed covering **launch sites, payloads, success rates, rankings, and time**:

- **Sites:** DISTINCT LaunchSite → 3 sites
- **Payloads:** total mass for 0–10000 kg; details for 4000–6000 kg; average payload per orbit
- **Success-rate ranking:** KSC LC-39A 77.3% > VAFB SLC-4E 76.9% > CCAFS SLC-40 60.0%
- **Booster ranking:** B1049 6/6 landings successful, B1051 5/5, B1059 4/4
- **Time analysis:** 18 launches in 2017 at 83.3% success; 19 launches in 2020 at 84.2%

```
SELECT LaunchSite,
  COUNT(*) AS Attempts,
  SUM(Class) AS Successes,
  ROUND(100.0*SUM(Class)/COUNT(*),1)
    AS SuccessRate
FROM SPACEXTBL
GROUP BY LaunchSite
ORDER BY SuccessRate DESC;
→ KSC LC 39A | 22 | 17 | 77.3
→ VAFB SLC 4E | 13 | 10 | 76.9
→ CCAFS SLC 40 | 55 | 33 | 60.0
```

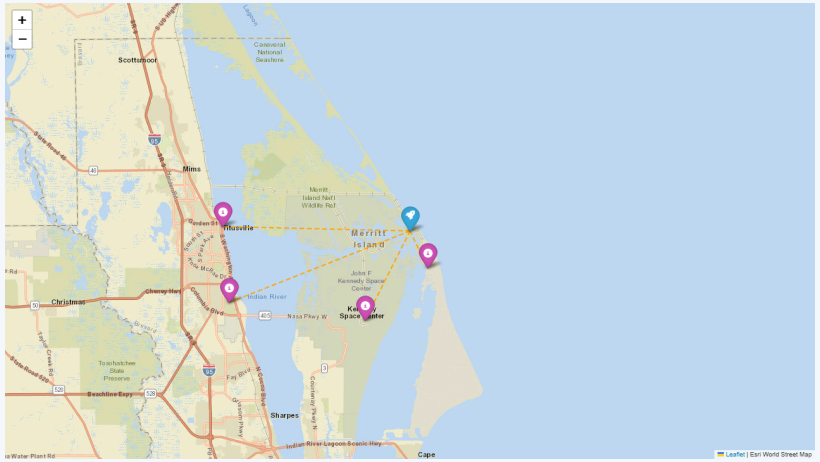
Full queries and results: sql/sql_results.txt (8 query groups)

🔗 GitHub: https://github.com/cho1eeee/spacex-capstone — scripts/05_sql_analysis.py · sql/sql_results.txt

Interactive Visual Analytics ①: Folium Launch-Site Maps and Proximity Analysis



① Launch-site markers: KSC LC-39A / CCAFS SLC-40 (Florida) and VAFB SLC-4E (California); green/red circles show success/failure landing counts

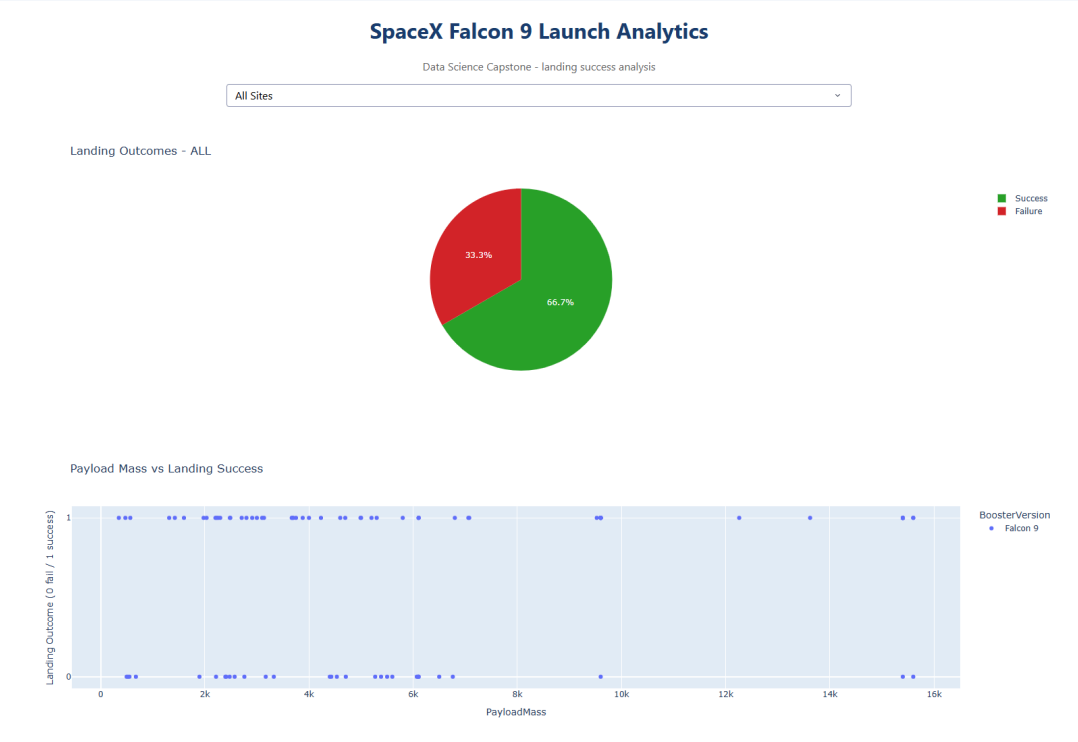


② KSC LC-39A proximity analysis (dashed lines are distance connectors)

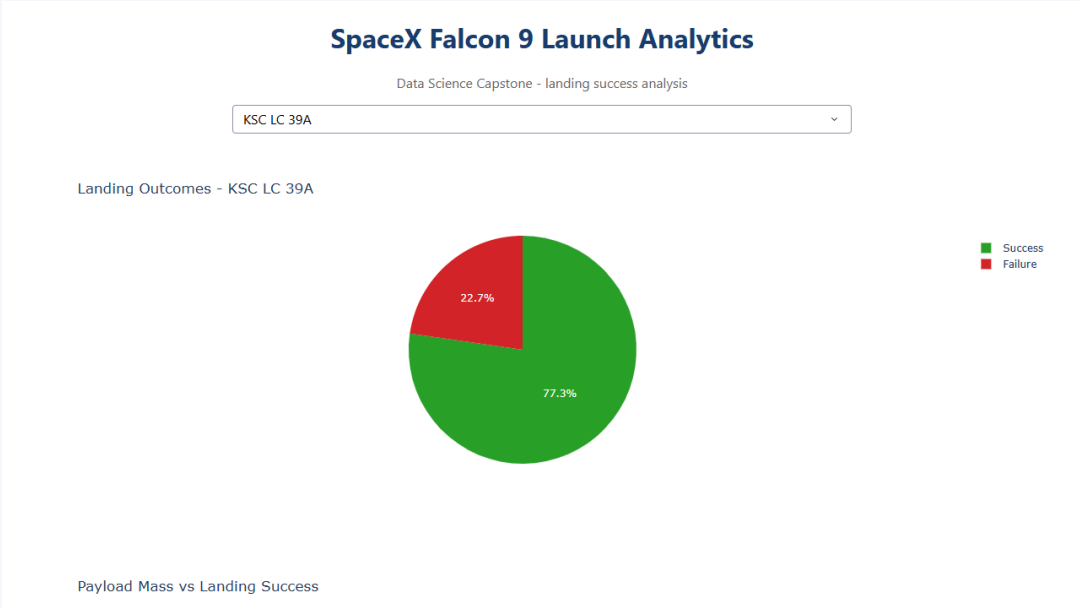
Landmark	Distance
Coastline (nearest)	10.57 km
Railway	4.33 km
Highway FL-401	20.59 km
Nearest city, Titusville	19.87 km

Launch-records map (launch_records_map.html) marks each launch and landing outcome with green/red circles; the proximity analysis shows sites hugging the coastline and railway — a key locational advantage for rocket transport and droneship deployment at sea.

Interactive Visual Analytics ②: Plotly Dash App



① All sites: landing-success pie (66.7% / 33.3%) + payload vs outcome scatter



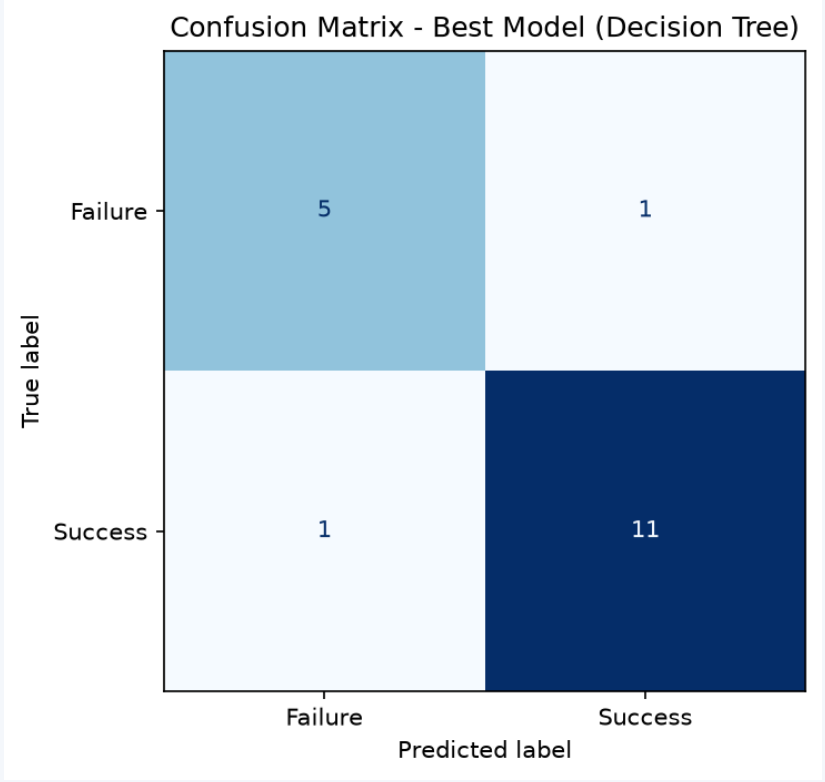
② Dropdown interactive filter: KSC LC 39A view (77.3% success rate)

- Components: a site dropdown (All Sites / CCAFS SLC 40 / KSC LC 39A / VAFB SLC 4E) linked to both charts.
- Callbacks: @app.callback responds to dropdown.value, recomputing and redrawing the pie and scatter in real time.
- Run locally with `python scripts/07_dash_app.py` → http://127.0.0.1:8050 (screenshots above show the app running successfully).

Predictive Analysis: Multi-Model Comparison, Evaluation and Confusion Matrix

Model	Best CV	Test accuracy	Key hyperparameters
Decision Tree (best)	0.807	**0.889**	entropy, max_depth=2
SVM (RBF)	0.846	0.833	C=10, gamma=scale
KNN	0.861	0.833	k=7, distance
Logistic Regression	0.804	0.778	C=10, L2
Random Forest (extra)	0.779	0.778	300 trees

Method: 80/20 stratified split + StandardScaler + 10-fold cross-validated GridSearchCV tuning.



Confusion matrix of the best model (Decision Tree): TN=5, FP=1, FN=1, TP=11

🔗 GitHub: https://github.com/cho1eeee/spacex-capstone — scripts/08_ml_models.py · models/model_results.txt

Conclusions, Best-Model Explanation and Novel Insights

Best-model explanation. The Decision Tree with max_depth=2 wins at 88.9% test accuracy, reaching high precision with only 2–3 key features — recovery outcomes are dominated by a few strong predictors (booster version/reuse state, orbit, launch site). The tree is interpretable and auditable, well suited to safety-related decisions.

Conclusions.

- First-stage recovery success is predictable; a lightweight model reaches ~89% accuracy;
- KSC LC-39A and VAFB success rates are significantly higher than CCAFS SLC-40 (which hosted the early experimental launches);
- The success-rate improvement is driven by **technical iteration** (>84% in the Block 5 era), not by lighter payloads.

Novel / creative insights.

- **Doubly cross-validated data pipeline:** automatic fallback when the primary API source fails + independent live-API verification — more robust than a single source;
- **Quantified geography:** Folium quantifies site proximity to coastline/railway/cities, explaining the recovery economics of site selection;
- **Interpretability first:** a shallow decision tree outperforms complex models — "less is more" for this problem;
- **Future work:** incremental updates from the live SpaceX API; weather and downrange-distance features; SHAP-based feature attribution.

Resources and Reproduction Guide

GitHub repository (all code, data and plots)

https://github.com/cho1eeee/spacex-capstone

Repository structure

- scripts/ — 01 collection · 02 scraping · 03 wrangling · 04 EDA · 05 SQL · 06 Folium · 07 Dash · 08 modeling
- **notebooks/ — 8 executed Jupyter notebooks with outputs (01–08)**
- data/ · plots/ · maps/ · sql/ · models/ — artifacts of each stage
- README.md · requirements.txt

One-command reproduction

```
pip install -r requirements.txt
python scripts/01_collect_data.py
python scripts/02_scrape_wikipedia.py
python scripts/03_clean_data.py
python scripts/04_eda.py && python scripts/05_sql_analysis.py
python scripts/06_folium_maps.py && python
scripts/08_ml_models.py
python scripts/07_dash_app.py  # http://127.0.0.1:8050
```

Tech stack: Python · pandas · requests · BeautifulSoup · SQLite · Folium · Plotly Dash · scikit-learn